Principles of Robot Autonomy I

Camera calibration, PnP, SfM

Stanford University

SIA Lab
Safe and Intelligent Autonomy Lab

MSL

IPRL

ASL
Autonomous Systems Lab

Logistics

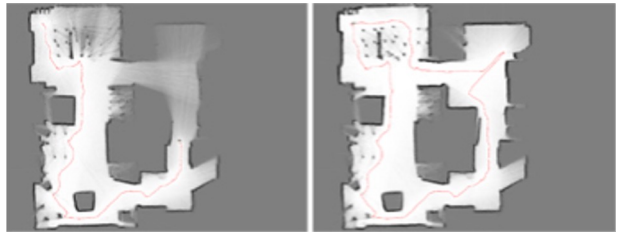
- Homework 3: due **Tues, Oct 27, at 11:59pm**
- Homework 1 grades released. Submit regrade requests by 10/25.
- Homework 4: out Tues Nov 3 (After the Midterm)
- Midterm: Thursday, Oct 29, in-class. Covers up to lecture 8.
- Lecture 8:
  - Camera calibration
  - N point perspective problem (PnP)
  - Structure from Motion (SfM)

Perception Stack: Computer vision, filtering, SLAM

Use sensor data to update the models

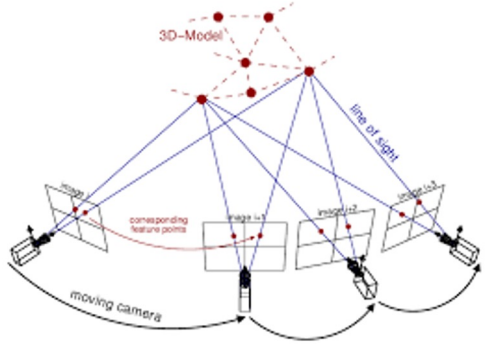
Localization, Mapping, Tracking:

- EKF/Monte Carlo localization
- Occupancy grid mapping
- Pose graph optimization
- Tracking (EKF and Particle Filter)
- **AA273: Filtering (Schwager)**
- **AA275: Navigation (Gao)**



Information extraction

- Computer vision: features, correspondences, Structure from Motion (SfM), depth
- Lidar scan matching, ICP
- **CS231A: Comp Vision**

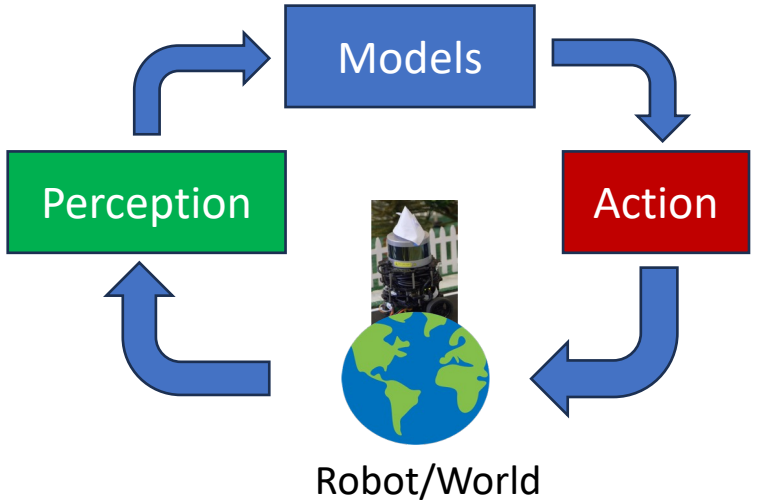


Sensors:

- RGB Cameras, RGB-D/stereo cameras, Lidar
- IMU, GPS, wheel encoders







Recall: Perspective projection in homogeneous coordinates

- Projection can be equivalently written in homogeneous coordinates

$$\begin{bmatrix} \alpha & 0 & u_0 & 0 \\ 0 & \beta & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} = \begin{pmatrix} \alpha X_c + u_0 Z_c \\ \beta Y_c + v_0 Z_c \\ Z_c \end{pmatrix}$$

K

Camera matrix/
Matrix of intrinsic parameters

P_c in homogeneous coordinates

Homogeneous pixel coordinates

- In homogeneous coordinates, the mapping is linear:

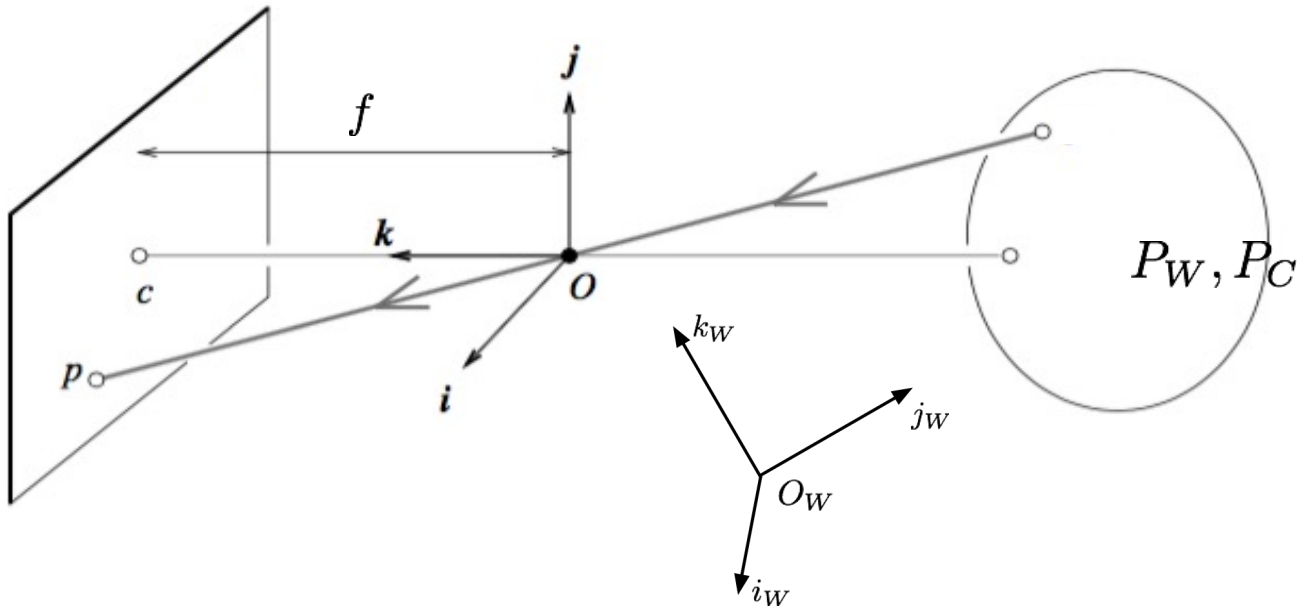
$$p^h = [K \quad 0_{3\times 1}] P_C^h$$

Point *p* in homogeneous pixel coordinates

Point P_c in homogeneous camera coordinates

Recall: Step 3

- Last step is to include in our mapping an additional transformation to account for the difference between the world frame and the 3D camera reference frame



Recall: Perspective projection equation

- Collecting all results

$$p^h = [K \quad 0_{3\times 1}]P_C^h = K[I_{3\times 3} \quad 0_{3\times 1}] \begin{bmatrix} R & t \\ 0_{1\times 3} & 1 \end{bmatrix} P_W^h$$

- Hence

Projection matrix *M*

$$p^h = K[R \quad t]P_W^h$$

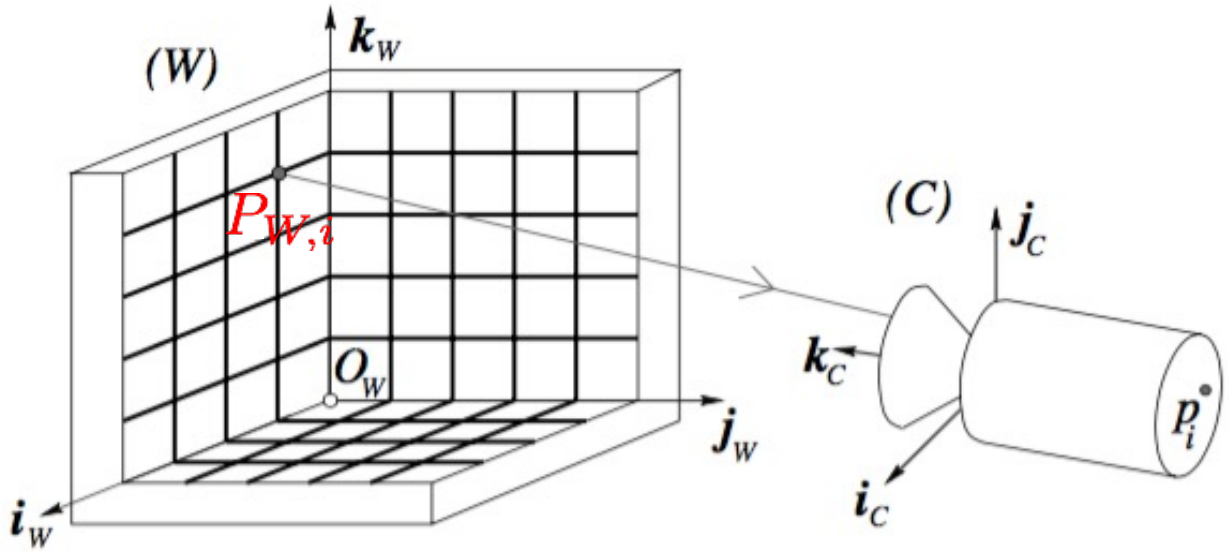
Intrinsic parameters

Extrinsic parameters (transform from world to camera, inverse of camera pose)

- Degrees of freedom: 4 for *K* (or 5 if we also include skewness), 3 for *R*, and 3 for *t*. Total is 10 (or 11 if we include skewness)

Camera calibration: direct linear transformation method

- Goal: find the intrinsic and extrinsic parameters of the camera

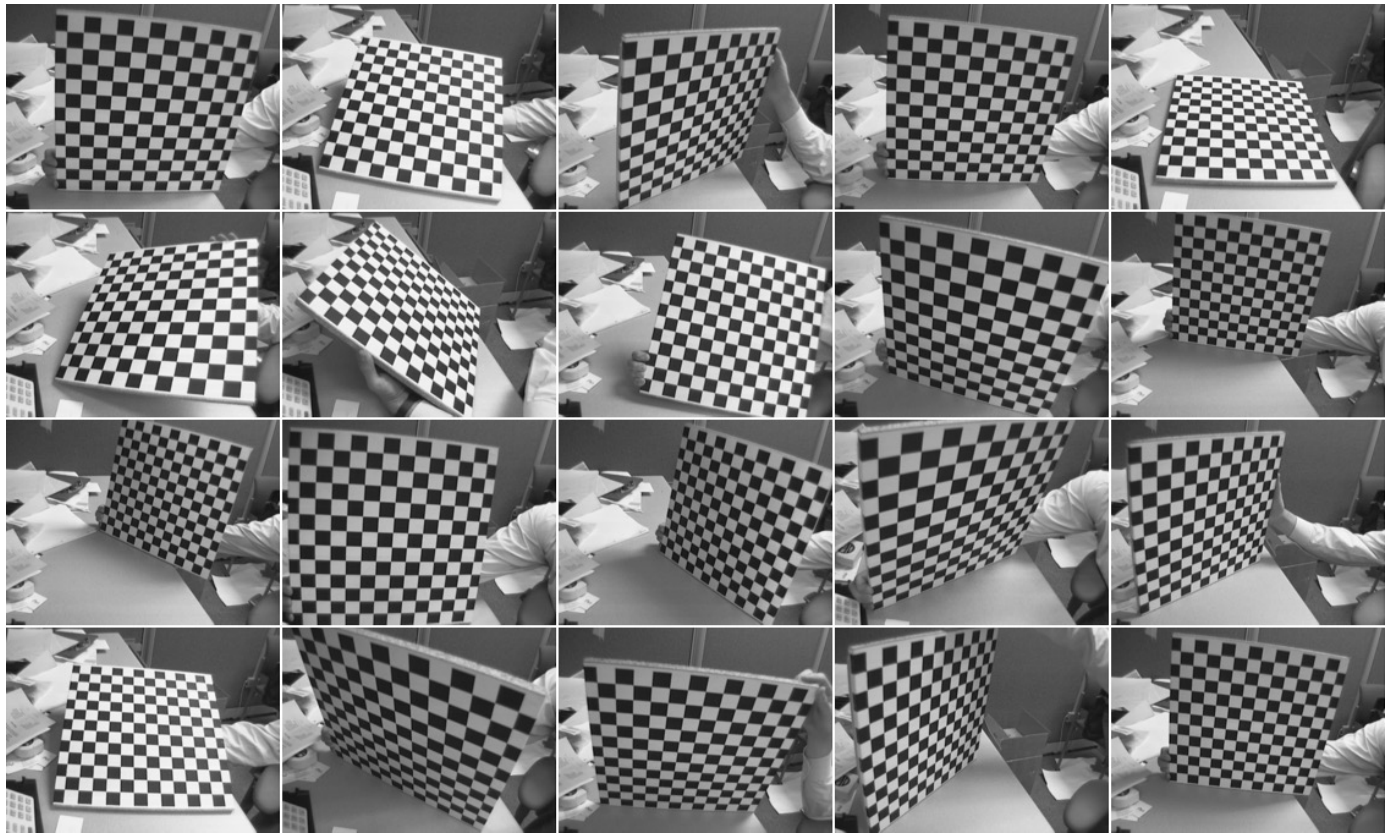


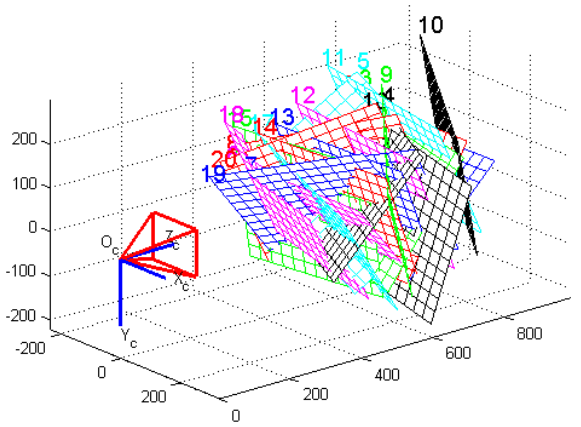
Strategy: given known correspondences $p_i \leftrightarrow P_{W,i}$, compute unknown parameters *K*, *R*, *t* by applying perspective projection

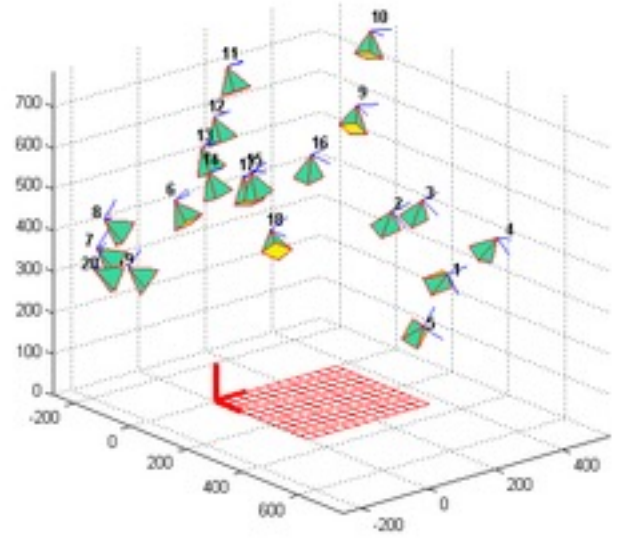
$P_{W,1}, P_{W,2}, \ldots, P_{W,n}$ with known positions in world frame

$p_1, p_2, \ldots, p_n$ with known positions in image frame

Examples for Calibration Images







- Due to co-planarity of cal board, need multiple images from different poses
- Origin of world frame defined by board lower right corner

Source: Wikipedia

Step 1

- First consider combined parameters

1×4 vector

$$p_i^h = M\, P_{W,i}^h,\ \ i = 1, \ldots, n, \qquad \text{where} \quad M = K[R \quad t] = \begin{bmatrix} m_1 \\ m_2 \\ m_3 \end{bmatrix}$$

- This gives rise to $2n$ component-wise equations, for $i = 1, \ldots, n$

$$u_i = \frac{m_1 \cdot P_{W,i}^h}{m_3 \cdot P_{W,i}^h} \qquad \text{or} \qquad u_i \left(m_3 \cdot P_{W,i}^h\right) - m_1 \cdot P_{W,i}^h = 0$$

$$v_i = \frac{m_2 \cdot P_{W,i}^h}{m_3 \cdot P_{W,i}^h} \qquad\qquad v_i \left(m_3 \cdot P_{W,i}^h\right) - m_2 \cdot P_{W,i}^h = 0$$

Calibration problem

- Stacking all equations together

$$\tilde{P}m = 0, \qquad \text{where } m = \begin{bmatrix} m_1^T \\ m_2^T \\ m_3^T \end{bmatrix}$$

$2n$ x 12 matrix of known coefficients

12 x 1 vector of unknown coefficients

12 x 1

- $\tilde{P}$ contains in block form the known coefficients stemming from the given correspondences
- To estimate 11 coefficients, we need at least 6 correspondences
- Due to co-planarity of cal board, need multiple images from different poses

Solution

- To find non-zero solution

$$\min_{m \in R^{12}} \quad \|\tilde{P}m\|^2$$
$$\text{subject to} \quad \|m\|^2 = 1$$

- Solution: select eigenvector of $\tilde{P}^T \tilde{P}$ with the smallest eigenvalue
- Readily computed via SVD (singular value decomposition)

Step 2

- Next, we need to extract the camera parameters, i.e., we want to factorize *M* as

$$M = [KR \quad Kt]$$

- Use "RQ" factorization (a variant of QR factorization): the submatrix $M_{1:3,1:3}$ is decomposed into the product of an upper triangular matrix *K* and a rotation matrix *R*.

- We compute t from $t = K^{-1}(Kt)$ where (Kt) is the last column of M.

- Calibration will be investigated in HW3

Perspective-n-Point (PnP) Problem (e.g. for object pose tracking)

Goal: find 3D object pose with respect to camera

- Known 3D coords of object points in its body frame, 2D image coordinates, and known correspondences
- Known camera calibration matrix K
- Same solution technique as camera calibration!

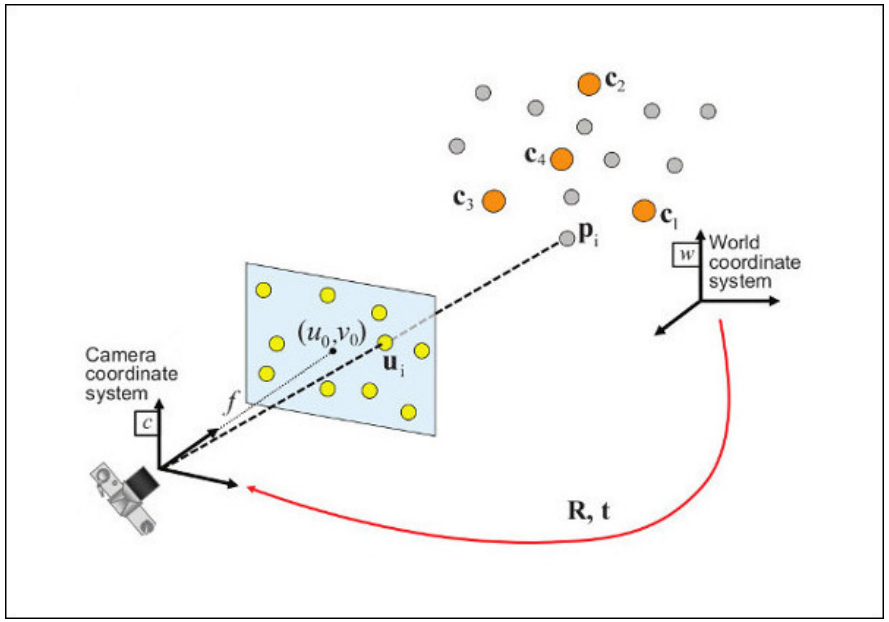
Pose of object wrt camera

$$K^{-1}p^h = [R \quad t]P_W^h$$

$$\min_{m \in R^{12}} \quad \|\tilde{P}m\|^2$$

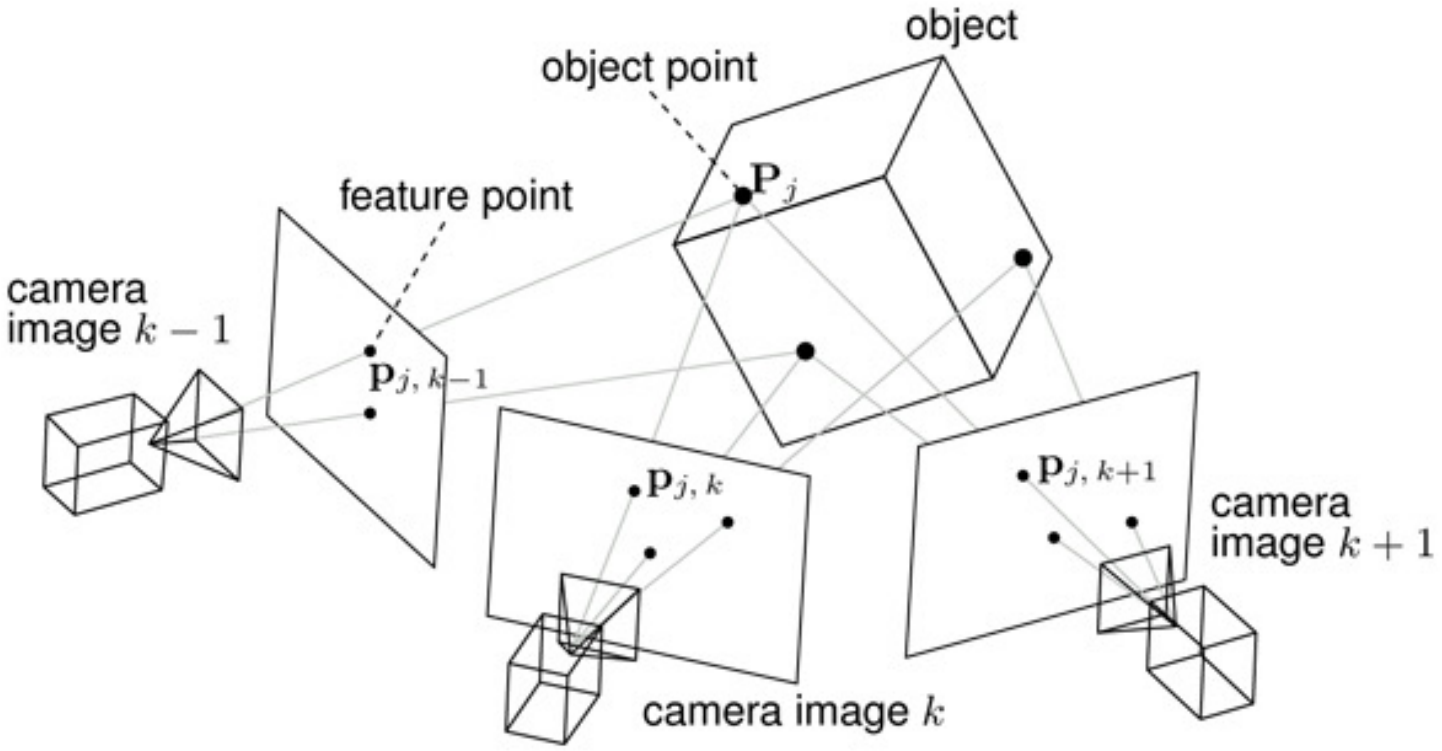
$$\text{subject to} \quad \|m\|^2 = 1$$

- $\tilde{P}$ and m defined with K^{-1} on LHS.



Solution: take SVD and find singular vector with smallest singular value.

Structure from motion (SFM)



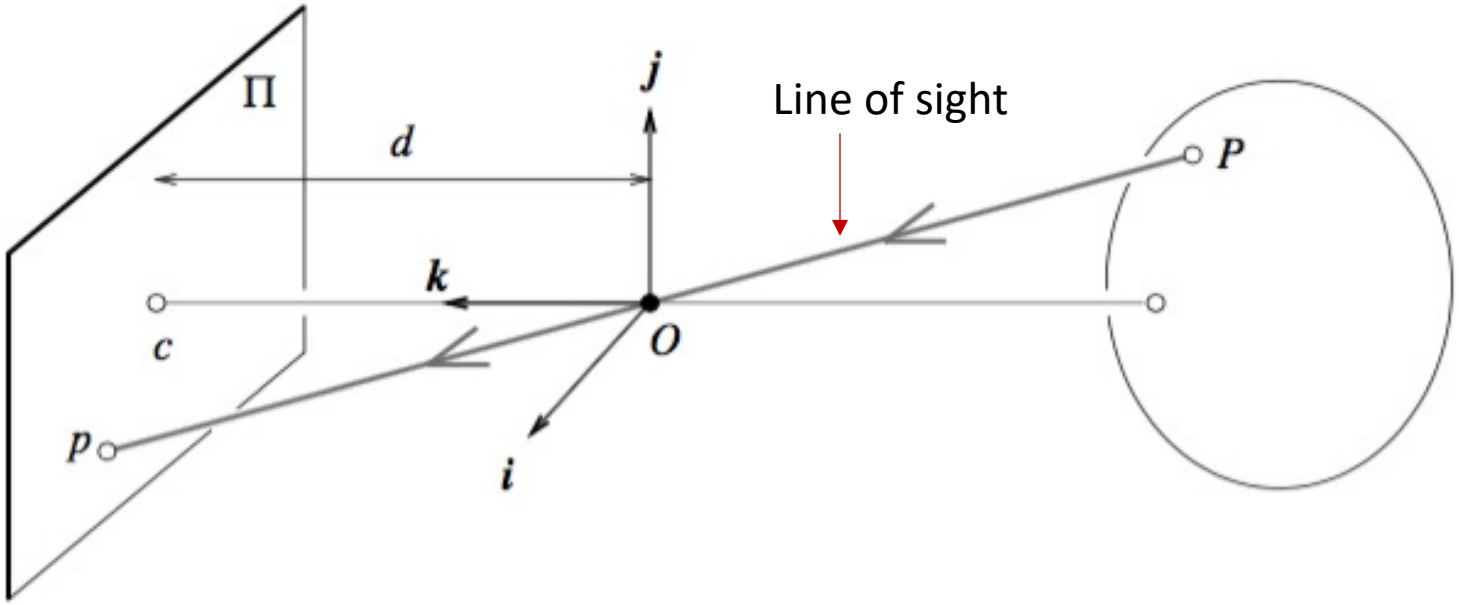
Given *m* images of *n* fixed 3D points

$$p_{j,k}^h = M_k P_j^h$$

Find:

- *m* projection matrices M_k (motion)
- *n* 3D points P_j (structure)

Measuring depth



$$p^h = K[R \quad t]P_W^h$$

Homogeneous coordinates

- Once the camera is calibrated, can we measure the location of a point *P* in 3D given its known observation *p*?
- No: one can only say that *P* is located *somewhere* along the line joining *p* and *O*!

Issues with recovering structure (3D point locations in world frame)

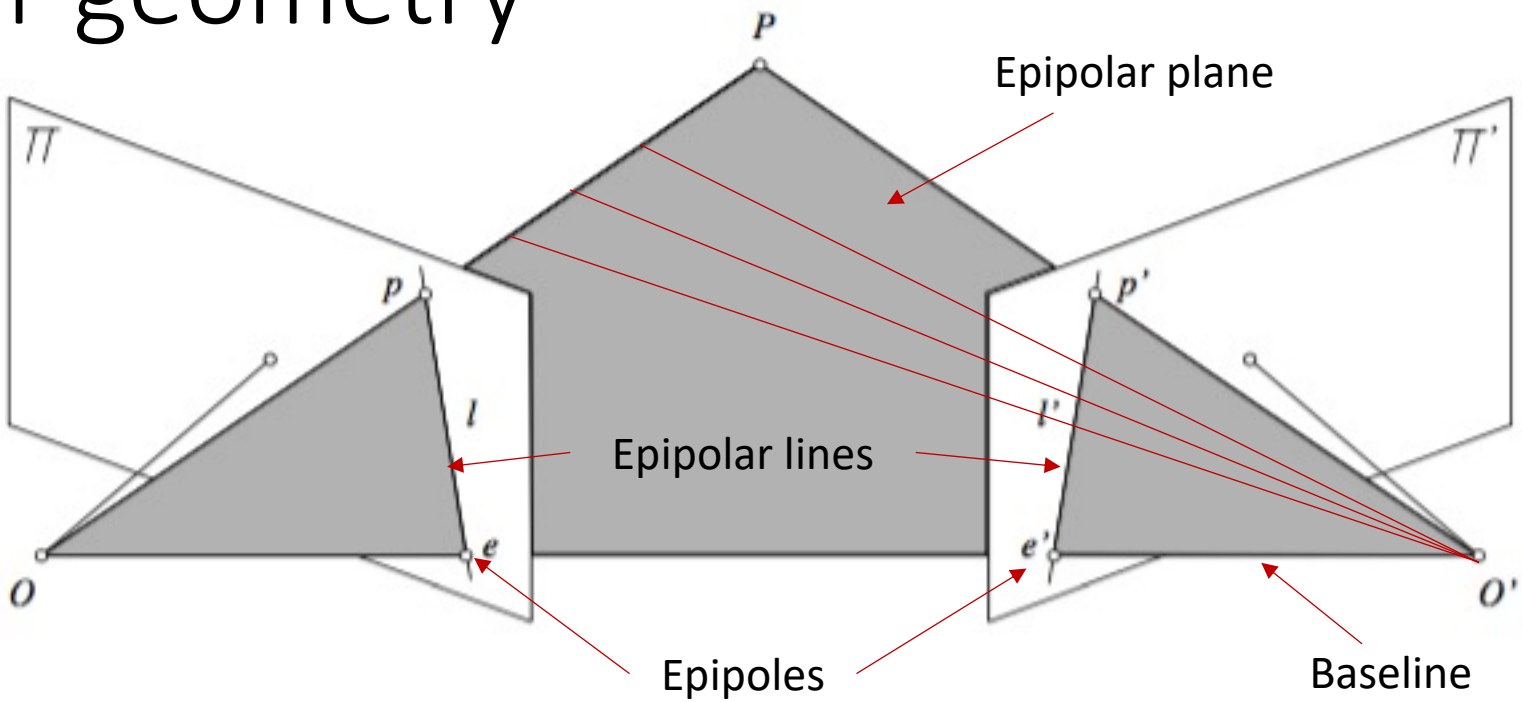


Recovering structure

- Structure: 3D point locations to be reconstructed by having access to only 2D images

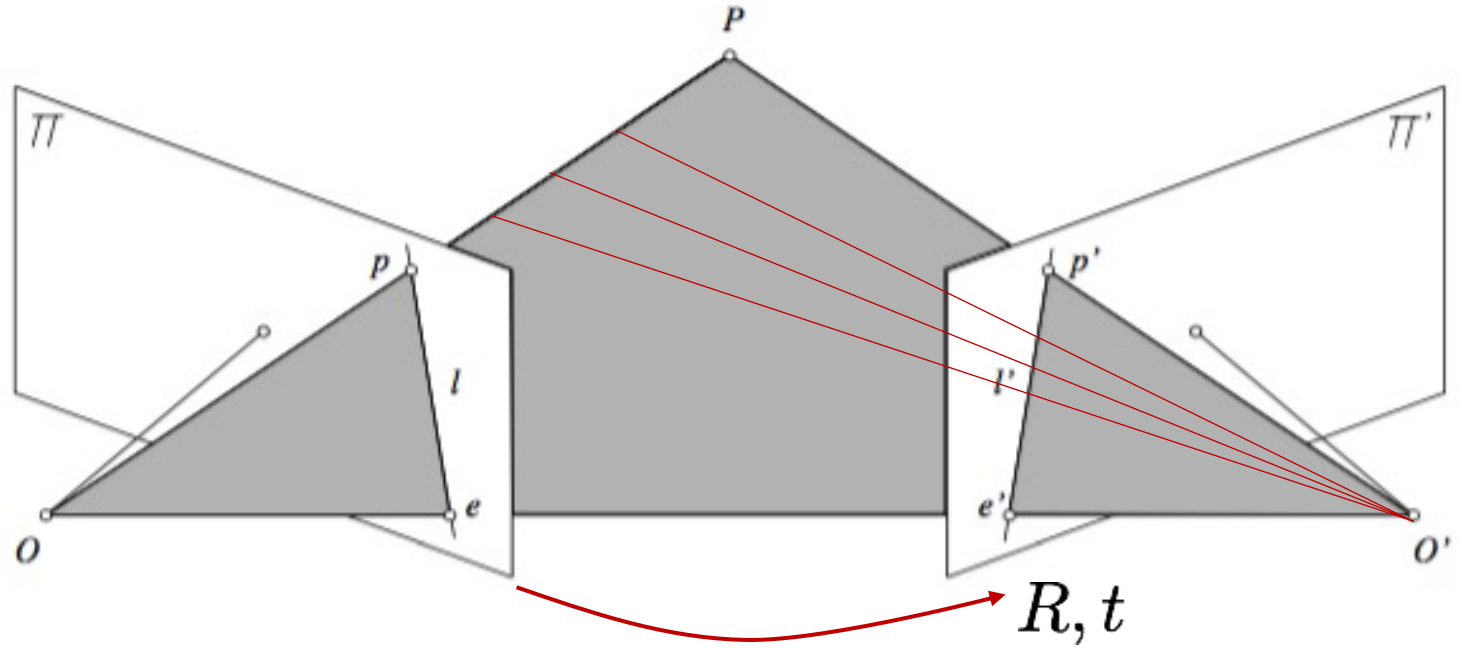
- Common methods
  1. Structure from motion (SfM): processes two images taken with the same or different cameras at *different times* and from different *unknown* positions
  2. Stereo vision: processes two distinct images taken at the *same time* and assumes that the relative pose between the two cameras is *known*
  3. Monocular depth estimation: Deep learned from context using a history of depth images (learned scale of objects, lighting queues, function queues, ect)
  4. Depth from focus: determines distance to one point by taking multiple images with better and better focus

Epipolar geometry



- Consider image coords p and p' of a point P in world frame observed by two cameras centered at O, O'
- These five points all belong to the *epipolar plane* defined by p, O, O', or equivalently, p', O, O'
- Epipolar constraint: potential matches for p must lie on epipolar line l' (and vice-versa)

Epipolar constraint derivation



- Epipolar constraint: $\overline{O'p'}$, $\overline{Op}$, and $\overline{O'O}$ (when expressed in that same frame) must be co-planar, or

$$\overline{O'p'} \cdot [\overline{O'O} \times \overline{Op}] = 0$$

Aside: matrix notation for cross product

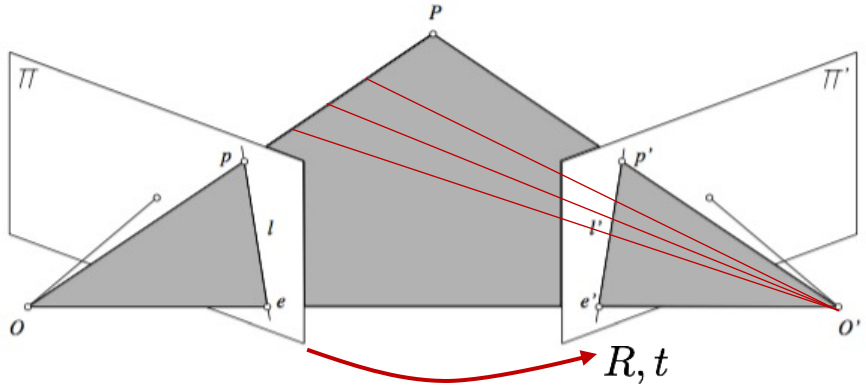
- Cross product can be expressed as the product of a skew-symmetric matrix and a vector

$$a \times b = \underbrace{\begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}}_{:= [a]_\times} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = [a]_\times b$$

Epipolar constraint derivation

(more details: CS231a)

- Translate into vector expressions, all in frame O’



$$\overline{O'p'} \cdot [\overline{O'O} \times \overline{Op}] = 0$$

$$(K'^{-1}p')^T [t]_\times (RK^{-1}p) = 0$$

$$p'^T F p = 0 \qquad \text{where} \qquad F = K'^{-T}[t]_\times RK^{-1}$$

Fundamental matrix

Key facts

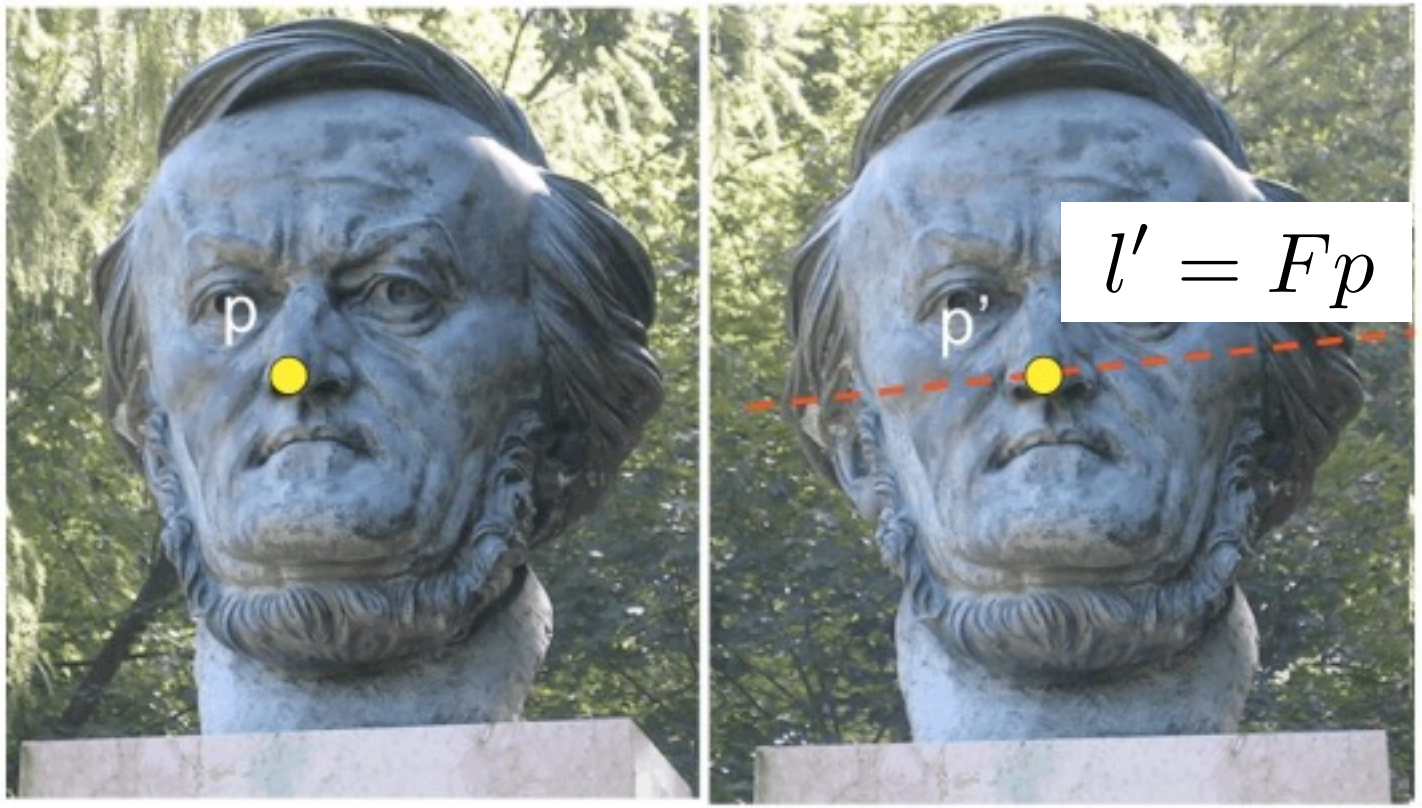
- F is referred to as the fundamental matrix

- $l' = Fp$ (resp. $l = F^T p'$) represents the epipolar line corresponding to the point p (resp. p') in the second (resp. first) image. This exploits the homogenous notation for lines.

- $Fe = F^T e' = 0 \rightarrow F$ is also singular (as t is parallel to the coordinate vectors of the epipoles)

- F has 7 DoF (9 elements – common scaling – det(F)=0)

Usefulness of fundamental matrix



- Assume *F* is given
- Given a point in image 1, one can compute the corresponding epipolar line in image 2 without any additional information needed!

Estimating the fundamental matrix

- 8-point algorithm

$$p = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \qquad p' = \begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix}$$

$$\Rightarrow \quad \begin{bmatrix} u' & v' & 1 \end{bmatrix} \begin{bmatrix} F_{11} & F_{12} & F_{13} \\ F_{21} & F_{22} & F_{23} \\ F_{31} & F_{32} & F_{33} \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = 0$$

$$\Rightarrow \quad \begin{bmatrix} u'u & v'u & u & u'v & v'v & v & u' & v' & 1 \end{bmatrix} \begin{bmatrix} F_{11} \\ F_{21} \\ F_{31} \\ F_{12} \\ F_{22} \\ F_{32} \\ F_{13} \\ F_{23} \\ F_{33} \end{bmatrix} = 0$$

f

*n*x9 matrix of known coefficients

$$\Rightarrow \quad Wf = 0$$

- Given $n \geq 8$ correspondences, one then solves

$$\min_{f \in R^9} \quad \|Wf\|^2 \quad \text{subject to} \quad \|f\|^2 = 1 \qquad \Rightarrow \tilde{F}$$

Enforcing the rank constraint

- $\tilde{F}$ satisfies the epipolar constraints, but is not necessarily singular (hence, is not necessarily a proper fundamental matrix)
- Enforce rank constraint (again, via SVD decomposition)

$$\text{Find } F \text{ that minimizes } \quad \|F - \tilde{F}\|^2$$
$$\text{subject to } \quad \det(F) = 0$$

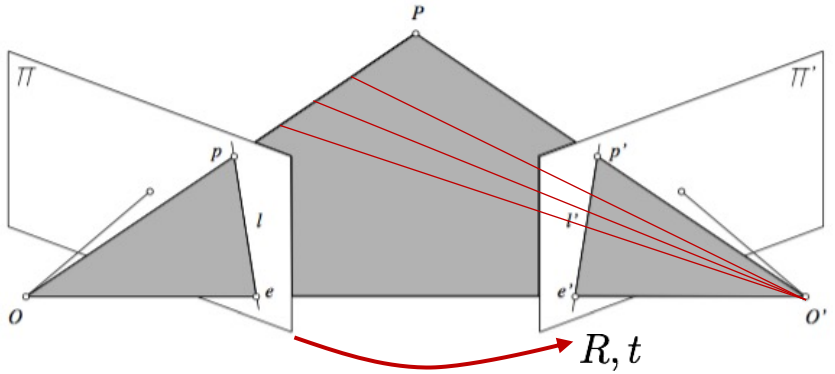
Frobenius norm

- 8-point algorithm
  1. Use least norms to compute $\tilde{F}$ by taking SVD of W, and finding right singular vector with smallest singular value.
  2. Enforce rank constraint on $\tilde{F}$ by taking SVD and setting smallest two singular values to zero, then re-multiply $\tilde{F} = U\Sigma V^T$

Obtaining relative camera pose

- Recall:

$$F = K'^{-T}[t]_\times R K^{-1}$$



- To get (R, t) we compute

$$[t]_\times R = K'^{T} F K$$

- Use RQ decomposition to get

$$[t]_\times R \rightarrow ([t]_\times, R)$$

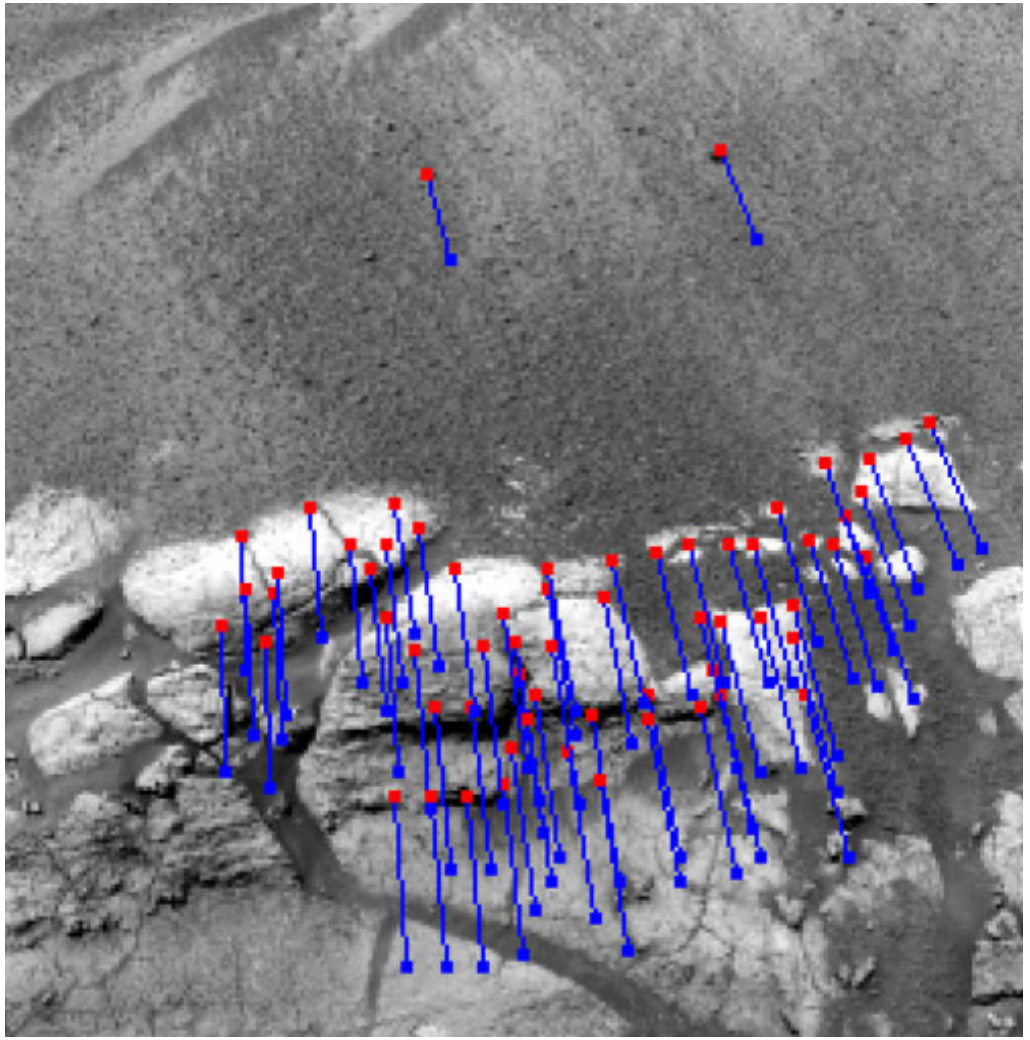
$$[t]_\times \rightarrow t$$

Solution to SFM problem (high-level)

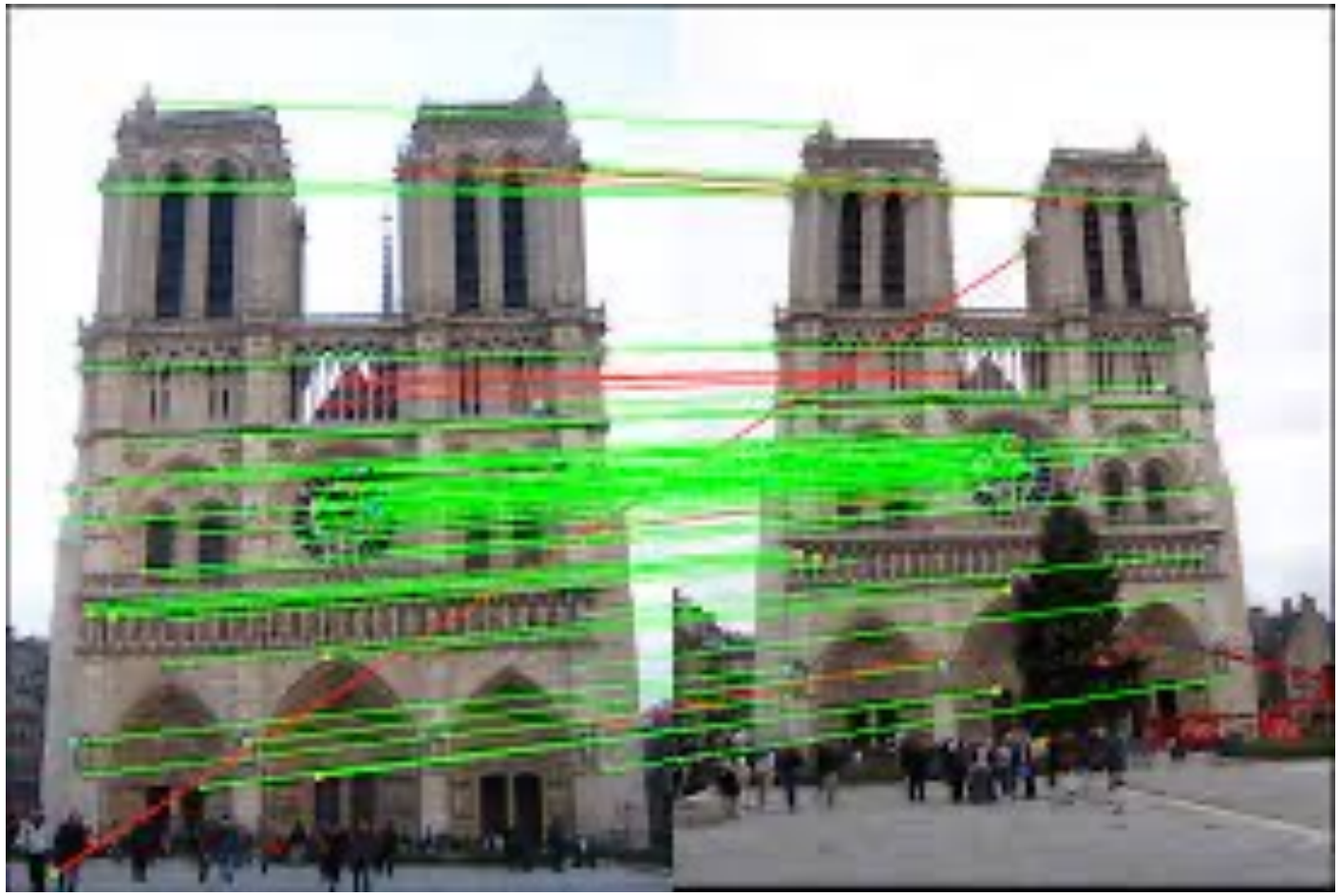
- Several approaches available:
  - Algebraic approach (by fundamental matrix)
  - Bundle adjustment (large-scale non-convex optimization problem to find [R t] camera poses for each image, and 3D points (i.e., structure) in common world frame
- Algebraic approach (2-views) – gives relative poses between two successive views, and 3D points relative to first view
  1. Compute fundamental matrix *F* (e.g., via 8-point algorithm)
  2. Use *F* to estimate camera projection matrices
  3. Use camera projection matrices for triangulation
- Used as ”front end” in visual SLAM, to obtain pose graph for PGO back end

Application of SFM: visual odometry

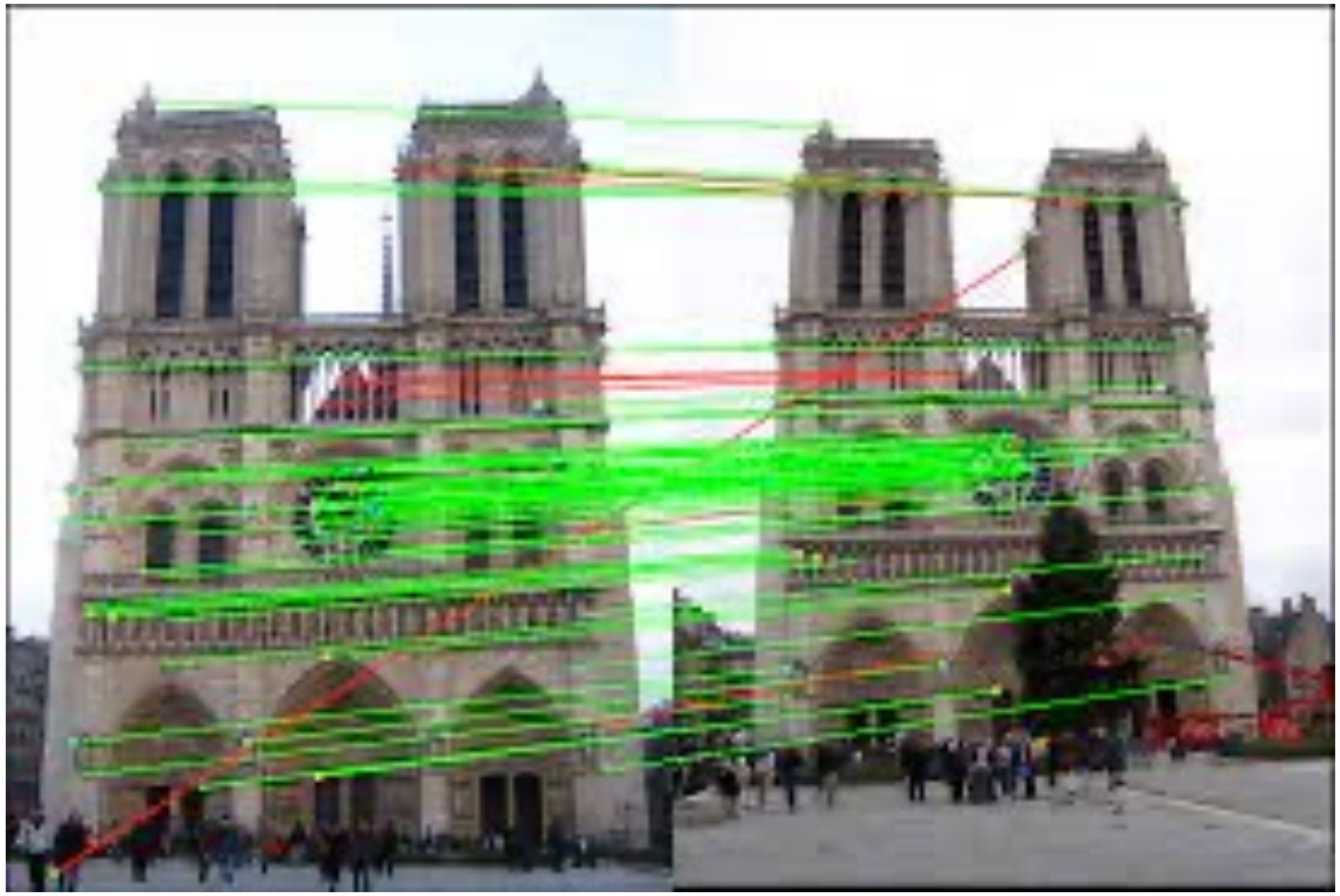
- SLAM/Visual Odometry: estimate the motion of the robot by using visual input (and possibly additional information)
  - Single camera: absolute scale must be estimated in other ways
  - Stereo camera: measurements are directly provided in absolute scale



Elephant in the room: how to find correspondences for PnP, stereo, or SfM?

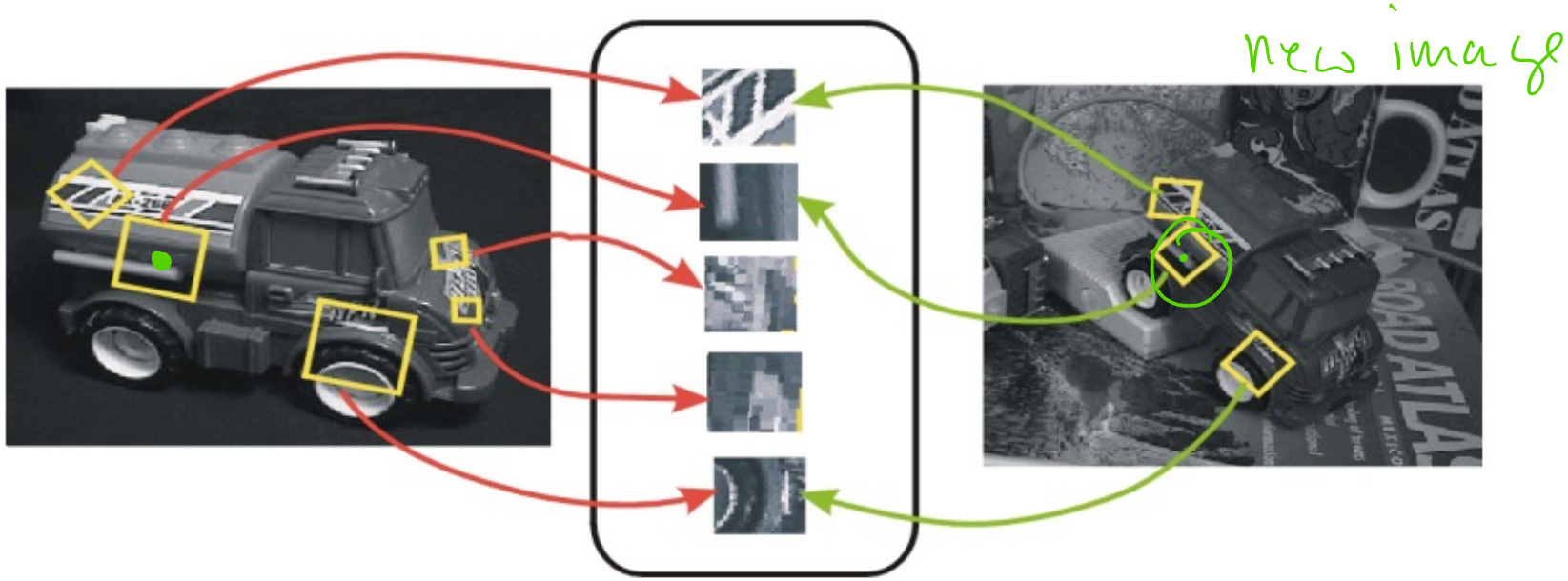


Features, feature descriptors, feature matching, RANSAC



Keypoint Features and their Descriptors

- Goal: *describe* keypoints so that we can compare them across images or use them for object detection or matching
- Desired properties:
  - Invariance with respect to pose, scale, illumination, etc.
  - Distinctiviness

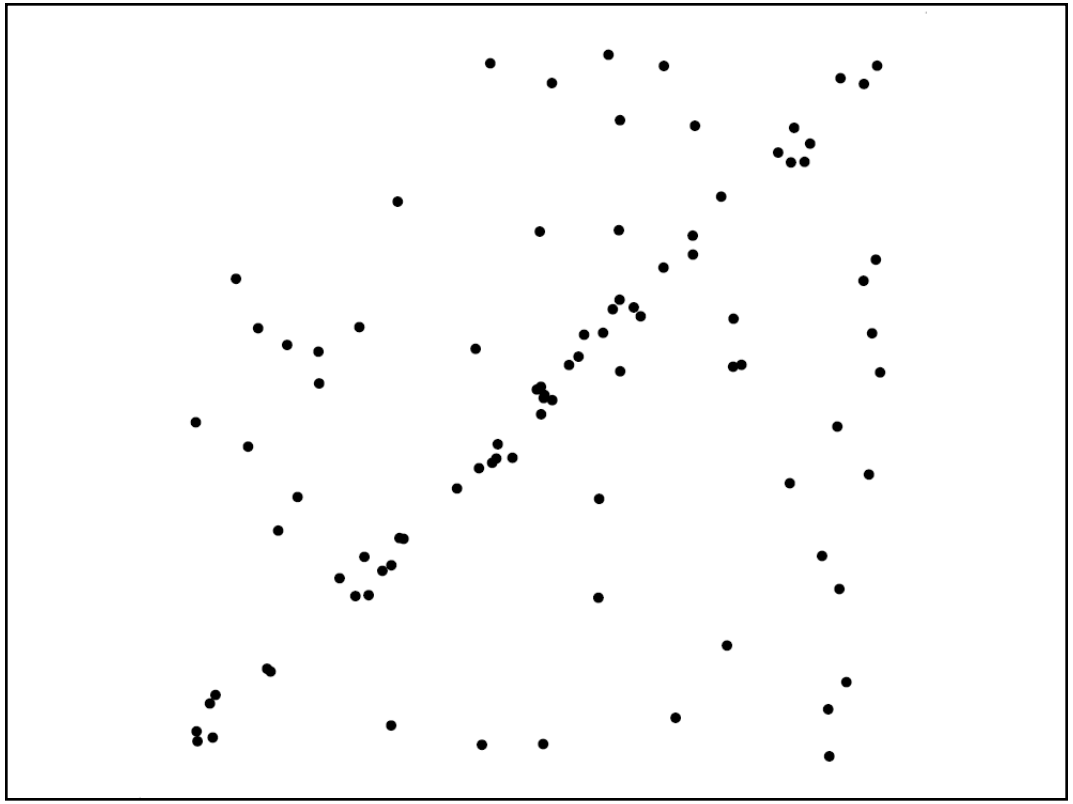


RANSAC iterations

- In principle, one would need to check all possible combinations of 2 points in dataset
- If $|S| = N$, number of combinations is $\frac{N(N-1)}{2}$ → too many
- However, if we have a rough estimate of the percentage of inliers, we do not need to check all combinations…

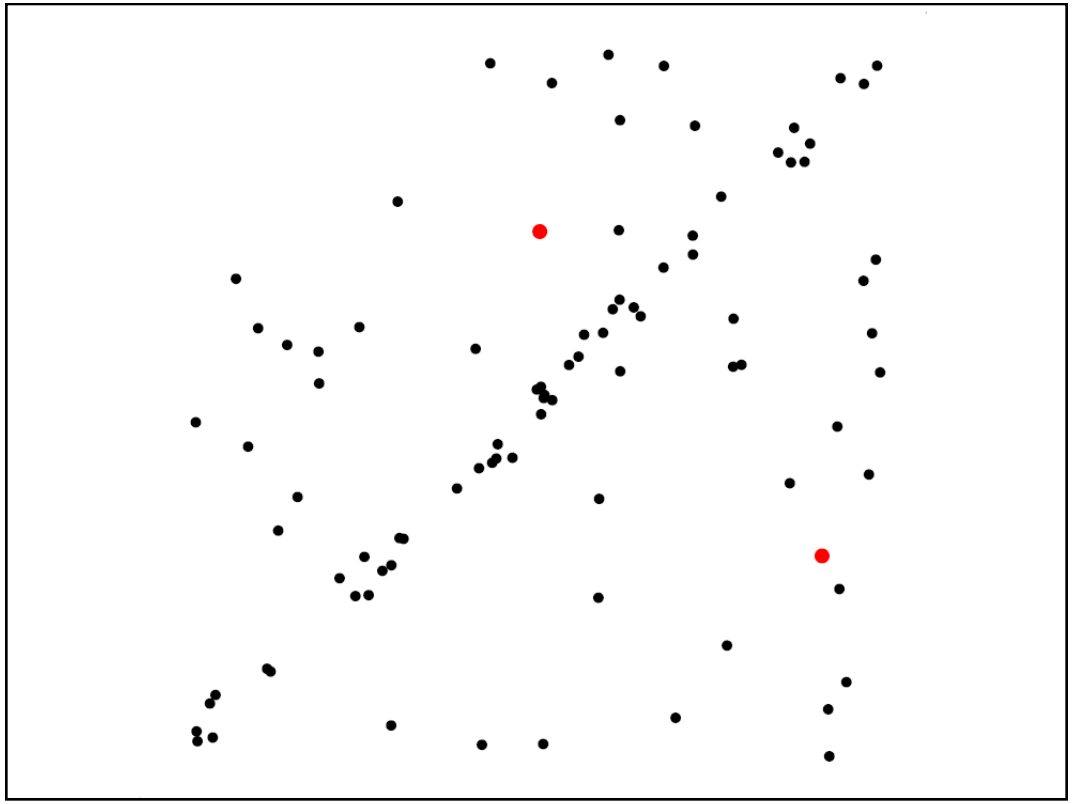
E.g., RANSAC for line fitting (for simplicity)

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



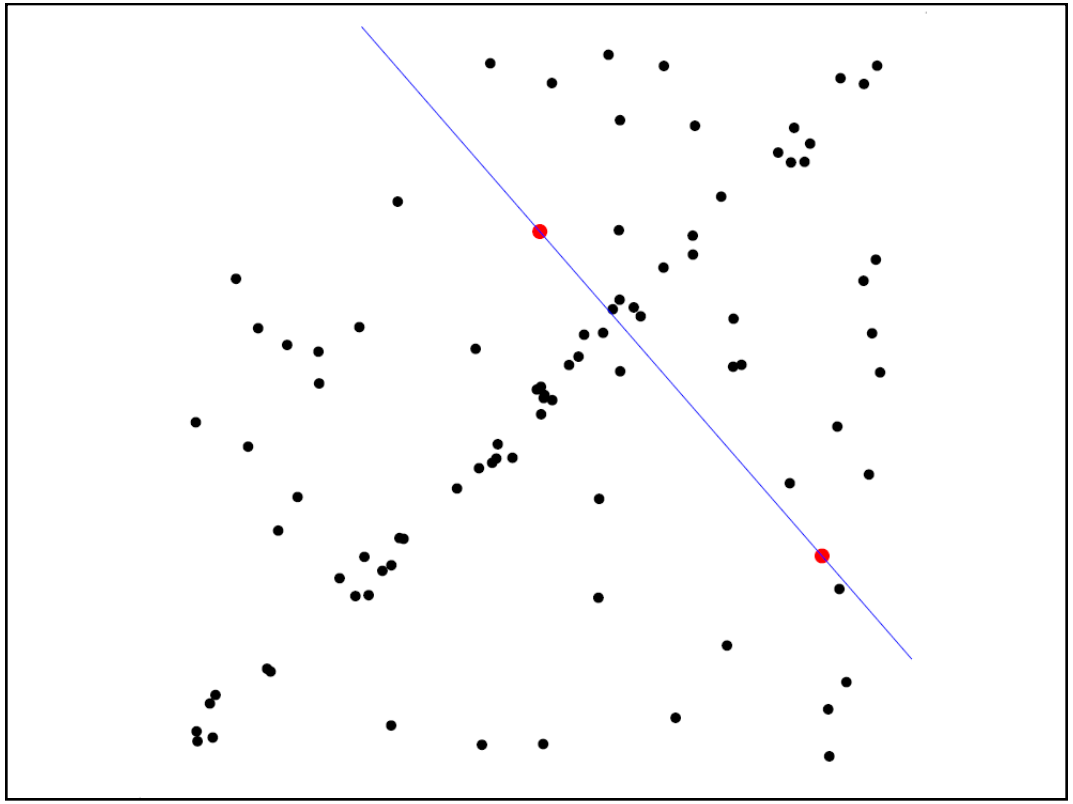
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



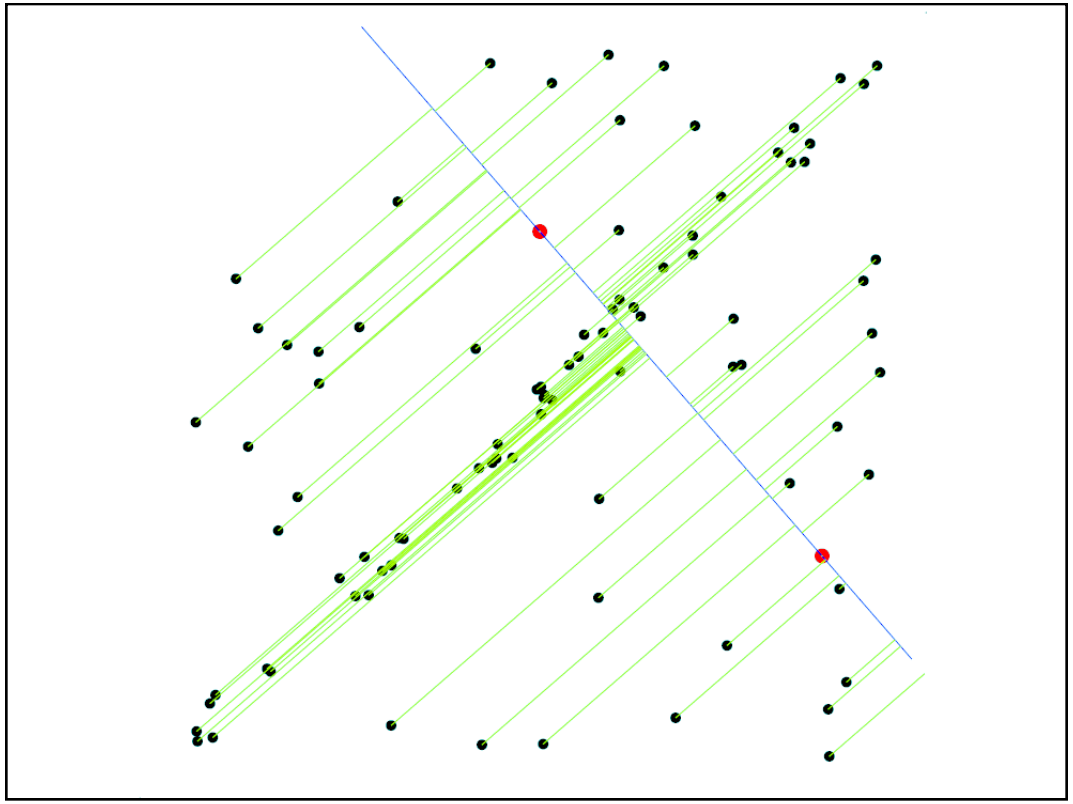
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



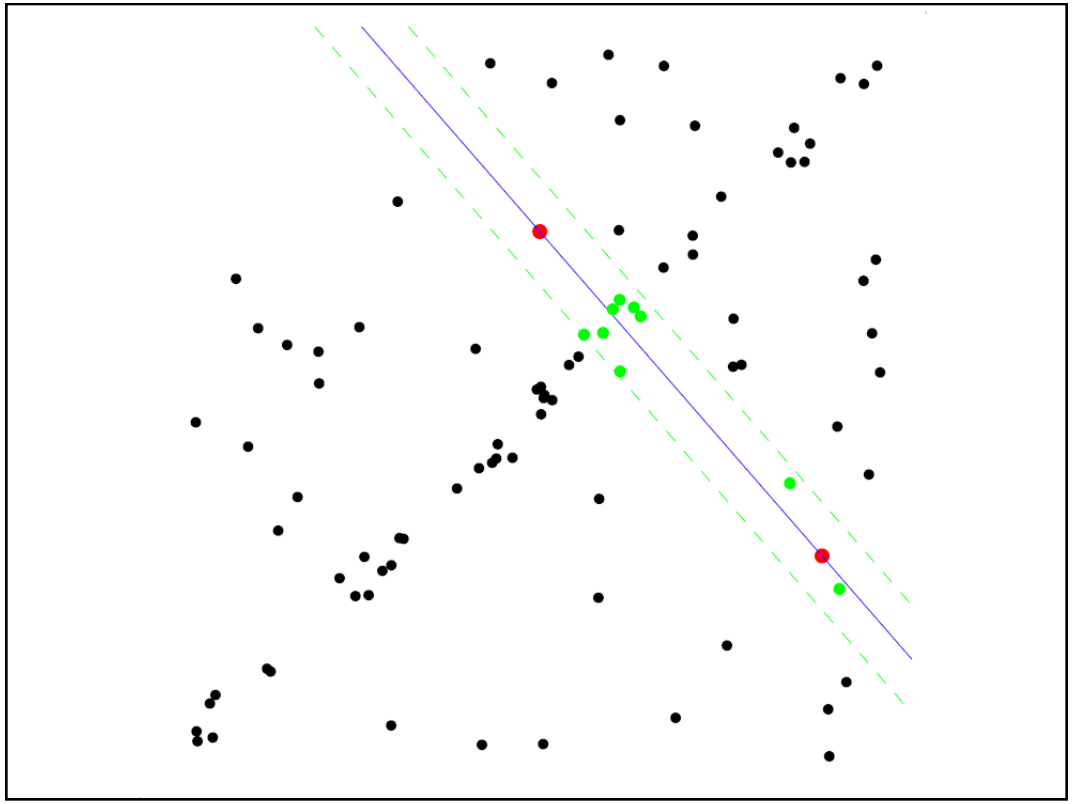
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



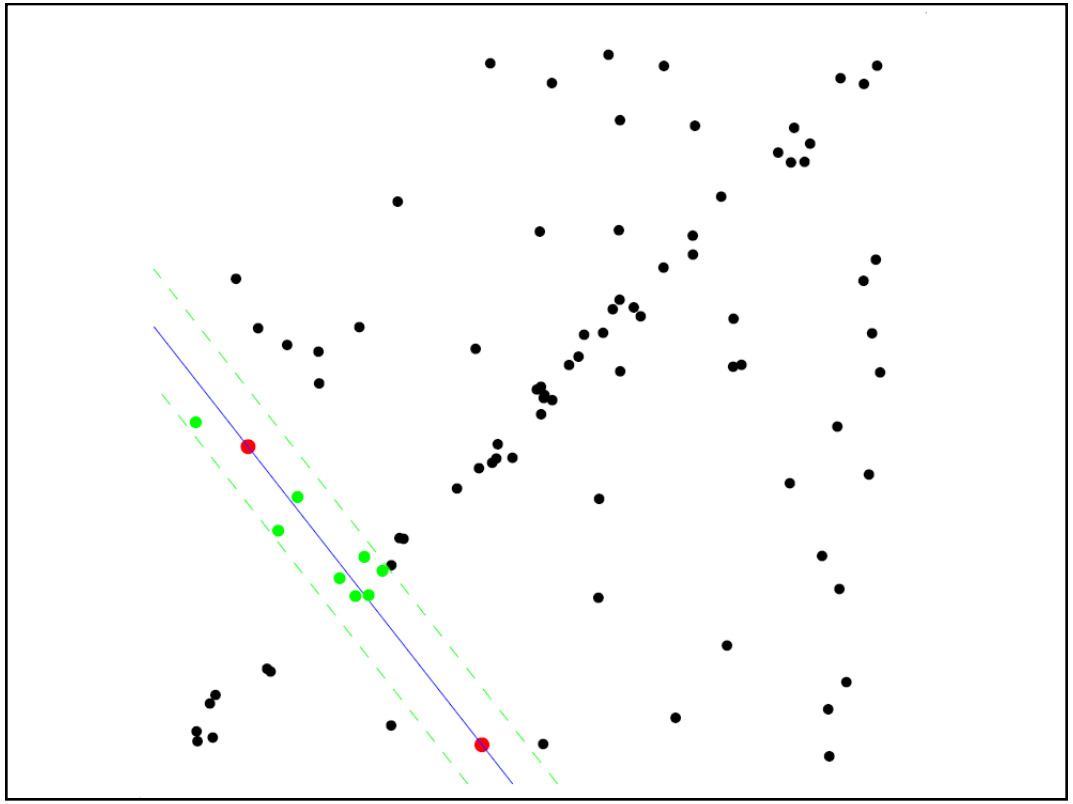
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



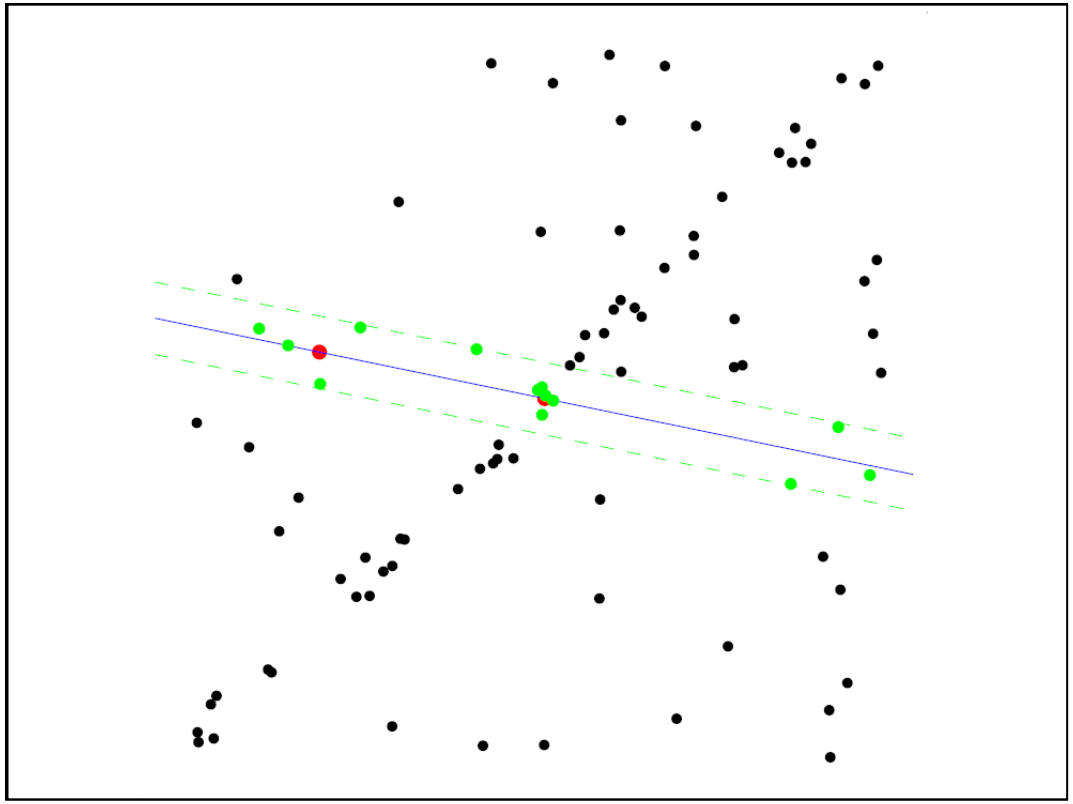
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



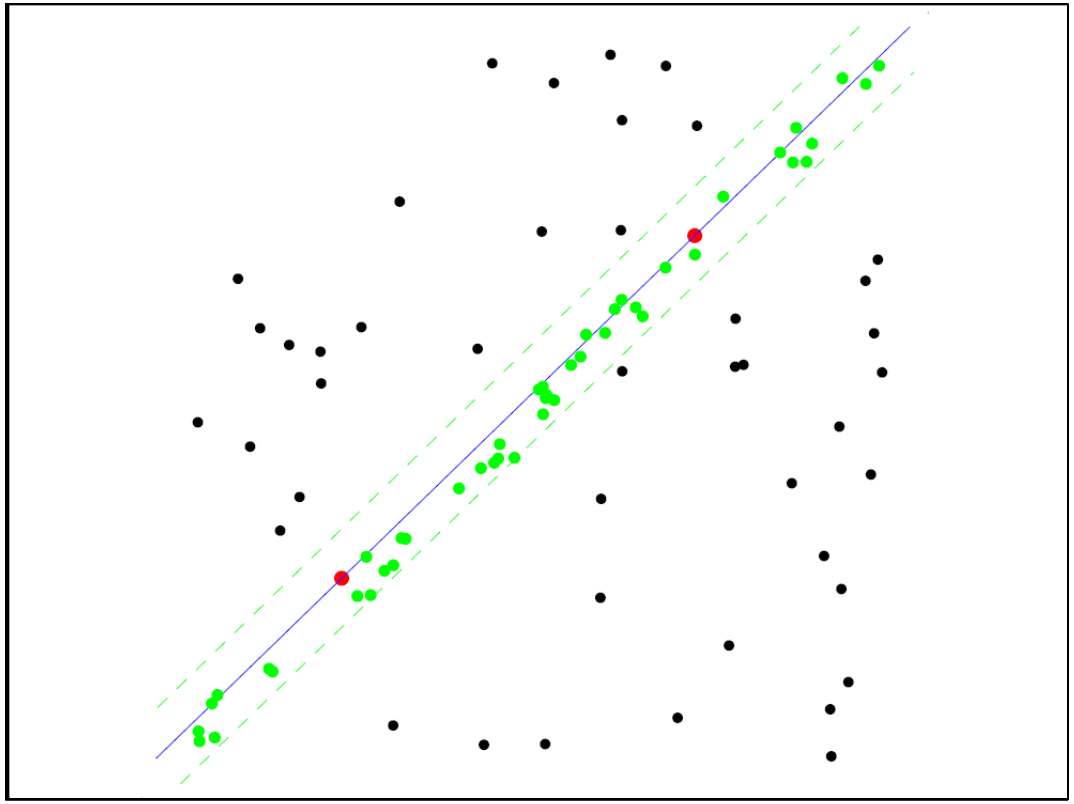
RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



RANSAC

Data: Set S consisting of all N points
Result: Set with maximum number of inliers
(and corresponding fitting line)
while $i \leq k$ **do**
randomly select 2 points from S;
fit line l_i through the 2 points;
compute distance of all other points to line l_i ;
construct *inlier* set, i.e., count number of
points with distance to the line less than γ;
store line l_i and associated set of inliers;
$i \leftarrow i + 1$
end
Choose set with maximum number of inliers



RANSAC iterations

- In principle, one would need to check all possible combinations of 2 points in dataset
- If $|S| = N$, number of combinations is $\frac{N(N-1)}{2}$ → too many
- However, if we have a rough estimate of the percentage of inliers, we do not need to check all combinations…

RANSAC iterations: statistical characterization

- Let w be the percentage of inliers in the dataset, i.e.,

$$w = \frac{\text{number of inliers}}{N}$$

- Let p be the desired probability of finding a set of points free of outliers (typically, $p = 0.99$)
- Assumption: 2 points chosen for line estimation are selected independently
  - $P(\text{both points selected are inliers}) = w^2$
  - $P(\text{at least one of the selected points is an outlier}) = 1 - w^2$
  - $P(\text{RANSAC nevers selects two points that are both inliers}) = (1 - w^2)^k$

RANSAC iterations: statistical characterization

- Then minimum number of iterations $\bar{k}$ to find an outlier-free set with probability at least p is:

$$1 - p = (1 - w^2)^{\bar{k}} \Rightarrow \bar{k} = \frac{\log(1 - p)}{\log(1 - w^2)}$$

- Thus, if we know w (at least approximately), after $\bar{k}$ iterations RANSAC will find a set free of outliers with probability p
- Note:
  - $\bar{k}$ depends only on w, not on N!
  - More advanced versions of RANSAC estimate w adaptively

Core “feature based” CV pipeline

- Input two images (stereo or SfM), or image and 3D model (PnP)
- Feature detection, feature matching -> N corespondences
- Problem: in practice, we get loads of bad feature matches
- Use Random Sample Consensus (RANSAC) to reject outliers
- In RANSAC, we
  - Sample a small subset n of matched features
  - Solve geometric CV problem with these correspondences (PnP, stereo, SfM)
  - Check solution fit with remaining (N-n) feature matches – record fit score
  - Repeat
  - Return the fit with the highest score